

Paidiverpy: A Python package designed to create pipelines for preprocessing image data for biodiversity analysis

Tobias Ferreira¹, Mojtaba Masoudi¹, Audenhaege van Loic¹, Erik Orenstein¹, Colin Sauze¹, and Jennifer Durden¹

¹ National Oceanography Centre, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Paidiverpy is an open-source Python package for constructing reproducible image-preprocessing workflows for biodiversity research. It provides a configurable, layer-based pipeline for combining image conversion, enhancement, spatial processing, sampling, metadata management, and custom processing operations while preserving the relationship between imagery and associated contextual information. Workflows can be defined declaratively, executed locally or using parallel computing resources, inspected at intermediate stages, and reused across datasets. By integrating established image-processing libraries, biodiversity-oriented metadata standards, scalable execution, and multiple user interfaces within a common workflow framework, Paidiverpy aims to reduce the technical effort required to transform heterogeneous scientific imagery into documented, reproducible, analysis-ready datasets for ecological analysis and automated image annotation.

Statement of Need

Imaging technologies are increasingly used to observe biodiversity across marine, terrestrial, and laboratory environments. Advances in digital cameras, autonomous platforms, microscopy, and related systems enable image collection at previously impractical scales (Durden & others, 2016). However, reliable ecological analysis still depends on consistent and scientifically appropriate preprocessing.

Preprocessing is important because acquisition variability can affect the detectability and comparability of biological features. Common operations include format conversion, enhancement, geometric correction, image selection, sampling control, and metadata standardisation. Several characteristics make this challenging:

- Heterogeneous image sources.** Biodiversity datasets are collected using diverse instruments, platforms, and environmental conditions, resulting in variation in formats, resolution, and preprocessing requirements (Durden & others, 2016; Pettoirelli et al., 2014).
- Inconsistent metadata handling.** Metadata describing when, where, and how images were acquired may be distributed across files, tables, and sensor records. Standards such as image FAIR Digital Objects (iFDOs) improve interoperability and reuse (Schoening et al., 2022; Wilkinson & others, 2016).
- Complex preprocessing requirements.** Biodiversity imagery often requires multiple interacting operations whose selection and order depend on the dataset and research objectives (Schoening et al., 2022; Song et al., 2022). Poor choices can affect downstream interpretation (Schoening et al., 2016; Young et al., 2017).

- 41 ▪ Scalability and computational constraints. Large, high-resolution collections can make
42 preprocessing computationally demanding, particularly when methods and parameters
43 require repeated testing ([Pettorelli et al., 2014](#); [Song et al., 2024](#)).
 - 44 ▪ Repeatability and provenance. Recording processing steps, parameters, and metadata
45 transformations is important for reproducibility and reuse across datasets and research
46 groups ([Schoening et al., 2016, 2022](#); [Wilkinson & others, 2016](#)).
- 47 Paidiverpy addresses these requirements by explicitly representing preprocessing operations,
48 metadata, and workflow configuration. Rather than replacing existing image-processing
49 libraries, it provides an orchestration layer for applying them consistently within reproducible
50 biodiversity-image workflows.

51 State of the Field

52 Mature Python libraries already provide many relevant image-processing algorithms. OpenCV
53 offers computer-vision and image-processing methods ([OpenCV Library, n.d.](#)), scikit-image
54 integrates image processing with the scientific Python ecosystem ([Walt & others, 2014](#)), and
55 Pillow provides image loading, manipulation, and format conversion ([Pillow Library, n.d.](#)).
56 Paidiverpy builds on these libraries rather than reimplementing their functionality. However,
57 they mainly provide operations on individual images or arrays, leaving researchers to manage
58 workflow order, configuration, metadata, and consistent processing across collections.

59 Scientific workflow systems can provide more general mechanisms for defining reproducible
60 computational pipelines, while distributed-computing frameworks such as Dask provide scalable
61 execution. These systems address broader workflow and execution problems but do not
62 themselves provide a biodiversity-image model combining image preprocessing, scientific
63 metadata, image sampling, and biodiversity-specific workflow requirements.

64 Paidiverpy fills this gap by combining established image-processing tools within a domain-
65 oriented framework for defining, validating, and executing reproducible preprocessing workflows.

66 Software Design

67 Paidiverpy was designed around the separation of workflow orchestration from image-
68 processing algorithms. Rather than implementing a new image-processing library, it integrates
69 established tools and exposes their functionality through a common pipeline abstraction. This
70 choice reduces duplication of mature software while allowing preprocessing operations to be
71 represented consistently, combined into reusable workflows, and extended with domain or
72 project-specific algorithms.

73 The central abstraction is a sequence of processing layers. Operations are grouped
74 into components including OpenLayer, ConvertLayer, ColourLayer, PositionLayer,
75 SamplingLayer, and CustomLayer. A central Pipeline coordinates these layers and maintains
76 the dataset as it moves through the workflow. The layer architecture provides a common
77 lifecycle for otherwise heterogeneous operations and makes it possible to inspect, replace,
78 reorder or extend individual stages without rewriting the complete pipeline.

79 A second design decision is to make the workflow itself a portable research object. Pipelines can
80 be described using YAML configuration files rather than solely through imperative Python code.
81 Parameters and processing order can therefore be reviewed independently of the implementation,
82 version controlled alongside analyses, and reapplied to another dataset. The configuration is
83 validated before execution, reducing the likelihood that differences between runs arise from
84 silently missing or inconsistent parameters.

85 Metadata are treated as part of the scientific dataset rather than auxiliary files. Paidiverpy
86 supports tabular metadata, image metadata and iFDO-compatible representations ([Schoening
87 & others, 2022](#)), allowing contextual information to remain associated with images while

88 preprocessing steps are executed. This is particularly important for biodiversity imagery because
89 variables such as acquisition time, position, camera orientation and sampling conditions may
90 determine which images should be retained or how they should be processed.

91 Scalability is provided through optional integration with Dask, rather than by implementing a
92 bespoke parallel-computing system. The same high-level pipeline abstraction can therefore be
93 used for local execution and parallel processing. This design keeps domain-specific workflow
94 logic within Paidiverpy while delegating scheduling and parallel execution to an established
95 ecosystem.

96 Extensibility was another explicit requirement. The CustomLayer mechanism allows researchers
97 to introduce project-specific preprocessing algorithms while preserving the surrounding workflow,
98 configuration and metadata-management infrastructure. This is important because biodiversity
99 imagery is heterogeneous and no fixed collection of preprocessing operations is appropriate for
100 every acquisition system or ecological question.

101 Different interaction mechanisms are provided for different user communities. Paidiverpy can
102 be used directly as a Python library, through a command-line interface, or in containerised
103 environments. The repository also provides example datasets and notebooks covering pipeline
104 creation, modification, parallel execution, object-store access, custom algorithms, RAW image
105 processing, and metadata export and validation. These interfaces share the same underlying
106 processing model rather than implementing independent preprocessing systems.

107 Paidiverpy is developed openly on GitHub under the Apache-2.0 licence. Documentation
108 is provided through ReadTheDocs, while example datasets and executable notebooks allow
109 potential users to exercise the principal workflow features without first preparing their own
110 dataset. Installation is available through pip, with Docker images also provided for containerised
111 execution.

112 Research Impact Statement

113 Paidiverpy is being used within marine-imaging research workflows at the National
114 Oceanography Centre and has begun to be adopted by researchers beyond its original
115 development team. These applications span the preparation and standardisation of imagery
116 used for biodiversity analysis and automated image-classification workflows.

117 One external application is associated with the DEAL (DEcentrAlised Learning for automated
118 image analysis and biodiversity monitoring) project led by Plymouth Marine Laboratory. DEAL
119 develops collaborative machine-learning approaches for analysing plankton and seafloor imagery
120 and addresses the challenge of obtaining sufficiently large, standardised and labelled datasets
121 for automated biodiversity monitoring.

122 Paidiverpy is also being evaluated/used in connection with deep-frameX, an open-source tool
123 for extracting scientific frames from deep-sea video. deep-frameX generates self-describing
124 image frames with embedded metadata and iFDO manifests for downstream image-analysis
125 systems. This provides a complementary workflow in which frame extraction and scientific
126 metadata management can precede preprocessing and analysis with Paidiverpy.

127 Within the National Oceanography Centre, Paidiverpy is used for **XXXXXXXXXXXXXXXXXX**,
128 providing a common workflow for preprocessing **[benthic/plankton/pelagic/etc.]** imagery.
129 These internal research applications have also informed the development of example datasets
130 and the broader best-practice guidance for biodiversity image preprocessing.

131 The package is consequently intended to serve as reusable infrastructure between image
132 acquisition and downstream manual or automated interpretation, rather than software tied to
133 a single dataset or analysis.

Conclusion

Paidiverpy provides a flexible and extensible framework for preprocessing ecological image datasets. Its layer-based design, explicit workflow definitions, metadata integration, and support for parallel execution address key limitations in existing image-based biodiversity analysis workflows. The package enables users to construct transparent preprocessing pipelines and generate analysis-ready datasets for downstream ecological and AI-based applications.

The software is openly available and distributed with example datasets, pre-built Docker images, and validation tools to support adoption. By standardizing image preprocessing workflows and improving repeatability, Paidiverpy contributes a reusable software component for large-scale biodiversity monitoring and ecological research.

Acknowledgements

This project was supported by the UK Natural Environment Research Council (NERC) through the *Tools for automating image analysis for biodiversity monitoring (AIAB)* Funding Opportunity, reference code **UKRI052**.

References

- Durden, J. M., & others. (2016). Perspectives in visual imaging for marine biology and ecology. *Oceanography and Marine Biology: An Annual Review*, 54. <https://doi.org/10.1201/9781315368597>
- OpenCV library. (n.d.). <https://opencv.org/>.
- Pettorelli, N., Laurance, W. F., O'Brien, T. G., Wegmann, M., Nagendra, H., Turner, W., & Milner-Gulland, E. J. (2014). Satellite remote sensing for applied ecologists: Opportunities and challenges. *Journal of Applied Ecology*, 51(4), 839–848. <https://doi.org/10.1111/1365-2664.12261>
- Pillow library. (n.d.). <https://pillow.readthedocs.io/>.
- Schoening, T., Durden, J. M., Faber, C., Felden, J., Heger, K., Hoving, H.-J. T., Kiko, R., Köser, K., Krammer, C., Kwasnitschka, T., Möller, K. O., Nakath, D., Nass, A., Nattkemper, T. W., Purser, A., & Zurowietz, M. (2022). Making marine image data FAIR. *Scientific Data*, 9, 414. <https://doi.org/10.1038/s41597-022-01491-3>
- Schoening, T., Osterloff, J., & Nattkemper, T. W. (2016). RecoMIA—recommendations for marine image annotation: Lessons learned and future directions. *Frontiers in Marine Science*, 3. <https://doi.org/10.3389/fmars.2016.00059>
- Schoenning, T., & others. (2022). An interoperable framework for underwater imagery metadata. *Scientific Data*, 9, 414. <https://doi.org/10.1038/s41597-022-01491-3>
- Song, Y., Nakath, D., She, M., & Köser, K. (2022). Optical imaging and image restoration techniques for deep ocean mapping: A comprehensive survey. *PFG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science*, 90(3), 243–267. <https://doi.org/10.1007/s41064-022-00206-y>
- Song, Y., She, M., & Köser, K. (2024). Advanced underwater image restoration in complex illumination conditions. *ISPRS Journal of Photogrammetry and Remote Sensing*, 209, 197–212. <https://doi.org/10.1016/j.isprsjprs.2024.02.004>
- Walt, S. van der, & others. (2014). Scikit-image: Image processing in python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>

- 176 Wilkinson, M. D., & others. (2016). The FAIR guiding principles for scientific data management
177 and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- 178 Young, N. E., Anderson, R. S., Chignell, S. M., Vorster, A. G., Lawrence, R., & Evangelista,
179 P. H. (2017). A survival guide to landsat preprocessing. *Ecology*, 98(4), 920–932.
180 <https://doi.org/https://doi.org/10.1002/ecy.1730>

DRAFT